

**SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY**



SUBJECT: ADVANCED MACHINE LEARNING
(SUBJECT CODE:24SAL051)

AN INDIVIDUAL TASK REPORT ON
**“Comparative Analysis of Logistic Regression, Decision
Tree and KNN for Diabetes Prediction”**

Submitted in the partial fulfilment of the requirements for the Fifth semester

**BACHELOR OF TECHNOLOGY
IN
Artificial Intelligence & Machine Learning**

Submitted By,
Manvitha M
01SU24AI048

UNDER THE GUIDANCE OF
Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146

2026-27

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY

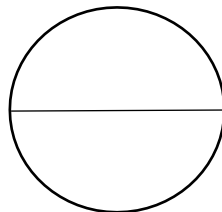


Department of Artificial Intelligence & Machine Learning

CERTIFICATE

This is to certify that, **Manvitha M Salvankar (01SU24AI048)**, has satisfactorily completed the assessment (Individual Task Report) in **Advanced Machine Learning (24SAL051)** prescribed by the Srinivas University for the 5th semester B. Tech. course during the year **2026-27**.

MARKS AWARDED



Staff In charge

Prof. Mahesh Kumar V B

Assistant Professor, Department of AIML

Date: 11/09/2026

TABLE OF CONTENTS

Sl.no	Description	Page no.
1	Abstract	1
2	Introduction	2
3	Dataset and Data Understanding	4
4	Data Preprocessing	6
5	Machine Learning Methodology	9
6	Implementation	11
7	Model Evaluation	19
8	Results and Analysis	21
9	Prediction and Discussion	22
10	Conclusion	23
11	References	24

Abstract

Diabetes is one of the major health challenges, and its early prediction can help in providing timely medical attention and improving medical decision-making. This project presents a machine learning-based approach for predicting diabetes using three classification algorithms: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). A diabetes dataset containing relevant medical and patient-related attributes is used to develop and evaluate the predictive models.

The dataset is first subjected to appropriate data preprocessing, including data inspection, feature selection, handling of missing values, encoding of the target class, feature scaling, and division into training and testing sets. The selected machine learning algorithms are then trained using the processed data, and their performance is evaluated using accuracy, precision, recall, and F1-score. The Logistic Regression model obtained approximately 75% accuracy, 65% precision, 67% recall, and 66% F1-score, while the Decision Tree and KNN models were evaluated using the same metrics for comparison.

The project also demonstrates the prediction of diabetes status for new patient data using the trained models. A comparative analysis of Logistic Regression, Decision Tree, and KNN helps identify differences in their classification performance and effectiveness. The results demonstrate that machine learning can be effectively applied to diabetes prediction and can provide useful computational support for analysing medical data. However, such predictions are intended for academic and decision-support purposes and should not replace professional medical diagnosis.

Among the three algorithms, the Decision Tree achieved the highest recall and F1-score, indicating a stronger ability to correctly identify patients with diabetes, while Logistic Regression offered a more interpretable and stable decision boundary with comparable accuracy. KNN, in contrast, showed relatively lower performance across all metrics, largely due to its sensitivity to feature scaling and the choice of the number of neighbors. These findings highlight that no single algorithm is universally superior, and model selection should be guided by the specific priorities of the application — such as favoring higher recall in medical screening contexts where missing a true positive case carries greater risk. The study underscores the potential of machine learning techniques to support early diabetes detection while also pointing to the need for further refinement, such as hyperparameter tuning, cross-validation, and larger, more balanced datasets, to improve the reliability and generalizability of the models

1.Introduction

Diabetes is one of the major health challenges affecting people worldwide. It refers to a chronic metabolic condition characterized by high blood glucose levels, which can significantly impact an individual's quality of life. Common factors associated with diabetes include unhealthy diet, obesity, lack of physical activity, family history, smoking, age, and genetic predisposition. Since several of these factors can occur together, analysing multiple patient characteristics can help identify patterns associated with diabetes.

Early prediction of diabetes is important because timely identification of potential risk can support further medical evaluation and appropriate treatment. Traditional diagnosis generally involves medical history, physical examination, laboratory tests, and other medical procedures. With the increasing availability of medical data, **Machine Learning (ML)** provides an additional computational method for analysing patient information. Machine learning algorithms can learn patterns from existing data and use these patterns to make predictions for new observations.

The development of an effective prediction model requires proper **data preprocessing**. Healthcare datasets may contain numerical and categorical attributes, different feature scales, missing or inconsistent values, and irrelevant information. Therefore, the dataset must first be inspected and prepared before applying machine learning algorithms. In this project, the data is processed through feature preparation, selection of required attributes, categorical encoding, feature scaling where necessary, and division into training and testing datasets. These steps help ensure that the machine learning algorithms receive suitable input data.

This project focuses on the application and comparison of three supervised machine learning classification algorithms: **Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN)**. These algorithms use different mathematical and computational approaches to classify patient records. Logistic Regression attempts to find the probability of belonging to each class. KNN uses probability-based classification, while Decision Tree creates a tree-like structure of conditions to reach a classification decision. Applying all three algorithms to the same dataset allows their performance to be analysed under similar conditions.

The project follows a systematic machine learning workflow beginning with dataset loading and understanding. The dataset is examined to identify its structure, attributes, and target variable. Exploratory analysis is performed to understand the distribution of the data and identify useful patterns. The required features are then prepared for machine learning by applying appropriate preprocessing techniques. After preprocessing, the dataset is divided into training and testing portions. The training data is used to build the models, while the testing data is used to evaluate their ability to classify previously unseen records.

Model evaluation is an important part of this project. Instead of considering only one performance measure, multiple evaluation metrics are used to obtain a more complete understanding of each algorithm. These include **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**. Confusion matrices and classification reports are also generated to analyse the

number of correctly and incorrectly classified samples. These measures allow the performance of Logistic Regression, Decision Tree, and KNN to be compared in a systematic manner.

In the implemented analysis, the **Logistic Regression model achieved an accuracy of approximately 51.00%**, with a precision of 65.00%, recall of 67.00%, F1-score of 66.00%, and ROC-AUC of 82.00%. These results indicate that Logistic Regression provided strong classification performance on the test data, particularly in identifying positive diabetes cases. The results of the other algorithms are also considered to determine their relative strengths and weaknesses and to identify the most suitable model for the given dataset.

The project further demonstrates the use of the trained models for predicting the outcome of **new patient data**. By providing previously unseen patient attributes to the trained models, the system generates a predicted class. This demonstrates the practical application of the developed machine learning pipeline and shows how classification algorithms can be used to analyse new observations after being trained on historical data.

Overall, this project demonstrates an end-to-end approach to diabetes prediction using machine learning. It covers dataset understanding, preprocessing, feature preparation, model development, performance evaluation, comparison, and prediction on new data. The comparison of Logistic Regression, Decision Tree, and KNN provides a clear understanding of how different classification techniques perform on the same medical dataset. The project also highlights the importance of selecting suitable features, preprocessing the data correctly, and using multiple evaluation metrics when developing predictive machine learning systems for medical applications. The developed models are intended for academic and analytical purposes and should not replace professional medical diagnosis or clinical judgement.

2.Dataset and Data Understanding

2.1 Pima Indians Diabetes Dataset

The dataset used in this project is the **Pima Indians Diabetes Dataset**. The objective of using this dataset is to analyse patient-related medical attributes and predict whether a patient is likely to have diabetes. The project applies three classification algorithms: **Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN)**.

The dataset contains different patient-related medical attributes that can be used to identify patterns associated with diabetes. These attributes include both numerical and categorical information. Before applying machine learning algorithms, the dataset is examined and prepared through appropriate preprocessing techniques.

The preprocessing process includes **checking for missing values, identifying categorical features, applying One-Hot Encoding, dividing the dataset into training and testing sets, and applying feature scaling**. These steps ensure that the data is converted into a suitable format for machine learning algorithms.

2.2 Dataset Features

The features in the dataset represent different characteristics of the patients that may be useful for predicting diabetes. Since the dataset contains different types of variables, each feature must be appropriately prepared before model training.

Numerical features contain measurable values, whereas categorical features represent different categories or groups. Categorical features are converted into numerical form using **One-Hot Encoding**. This allows the machine learning algorithms to process categorical information effectively.

Feature scaling is also applied as part of the preprocessing pipeline. Scaling is particularly useful for algorithms such as SVM because features with larger numerical ranges can otherwise have a greater influence on the model. The project therefore prepares the input data systematically before training the models.

2.3 Target Variable

The target variable represents the final output that the machine learning models need to predict. In this project, patients are classified into two categories:

- **No Diabetes**
- **Disease**

Therefore, the project is treated as a **binary classification problem**. The models learn patterns from the training data and use those patterns to predict the class of previously unseen patient records.

The distribution of the target classes is also analysed during exploratory data analysis. Understanding the distribution of the classes is important because an imbalance between classes can affect the interpretation of model performance.

2.4 Dataset Characteristics

The dataset contains a combination of medical attributes that provide information about the health characteristics of patients. These attributes are used as input variables for the machine learning algorithms.

Since the dataset contains numerical and categorical features, preprocessing plays an important role in the overall machine learning workflow. Missing-value checking, categorical feature identification, One-Hot Encoding, feature scaling, and train-test splitting are performed before model training.

After preprocessing, the resulting dataset is suitable for training the three selected classification algorithms. The same prepared data is supplied to **Logistic Regression, Decision Tree, and KNN**, allowing their performance to be compared under similar conditions.

2.5 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is performed before model development to understand the structure and distribution of the dataset. EDA helps identify important patterns and provides a better understanding of the relationship between the available attributes and the target variable.

One important analysis is the distribution of the target classes. The number of patients belonging to the **No Diabetes** and **Disease** categories is examined using graphical representation. Such analysis helps understand whether the target classes are reasonably distributed and provides useful information before model training.

Overall, the dataset understanding stage provides the foundation for the machine learning process. The available patient attributes are examined, the target variable is identified, and the data is prepared for subsequent modelling. After completing this stage, the processed dataset can be used for training and evaluating **Logistic Regression, Decision Tree, and KNN** classification models.

3. Data Preprocessing

3.1 Data Loading

The first step in developing the diabetes prediction system is loading the Pima Indians Diabetes Dataset into the Python environment. The dataset is imported into a Pandas DataFrame so that its records and attributes can be easily examined and processed. Loading the data correctly is important because all subsequent preprocessing and machine learning operations depend on the quality and structure of the input dataset.

After loading the dataset, its basic structure is examined to understand the number of records, number of features, data types, and target variable. This provides an initial understanding of the information available for building the prediction models.

3.2 Data Inspection

Data inspection is performed to identify the characteristics of the dataset before applying any transformation. The first few records can be displayed to understand how the features are represented. The shape of the dataset is also checked to determine the number of observations and attributes.

The data types of the different columns are examined because the preprocessing method depends on whether a feature is numerical or categorical. Numerical variables can generally be used after appropriate scaling, while categorical variables need to be converted into a numerical representation.

Data inspection also helps identify potential problems such as missing values, unexpected data types, duplicate records, or unsuitable attributes. Performing this step before model training reduces the possibility of errors during implementation.

3.3 Handling Missing Values

Missing values can negatively affect machine learning algorithms because most classification models require complete input data. Therefore, the dataset is checked for missing or null values before proceeding with model development.

If missing values are present, an appropriate treatment method can be applied depending on the nature of the feature. Numerical values may be handled using statistical measures such as the mean or median, while categorical values can be handled using the most frequent category or another suitable method.

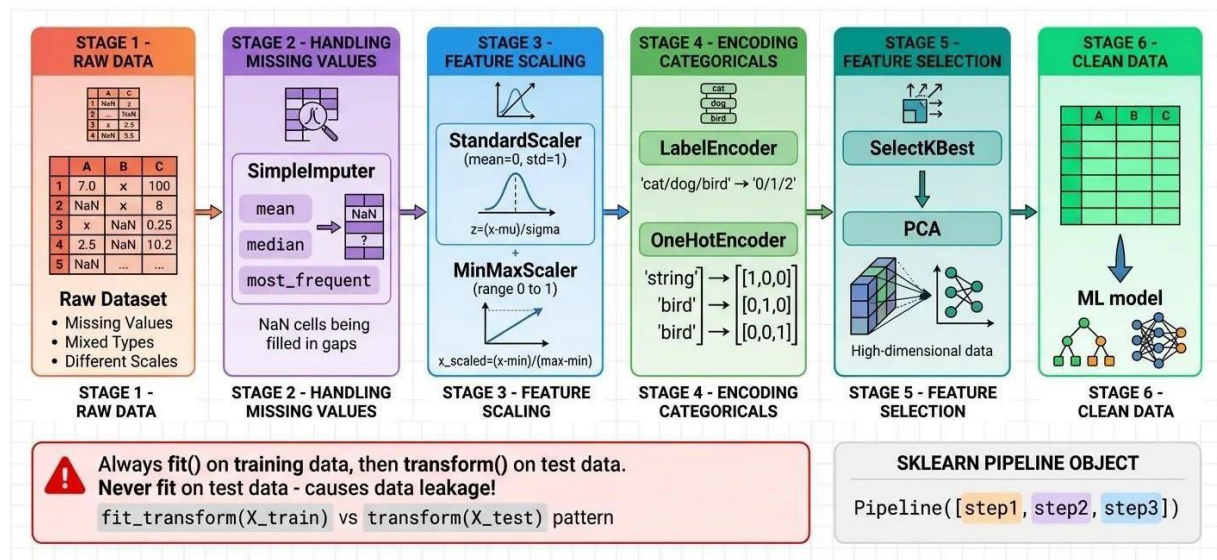
In this project, missing-value checking is included as part of the preprocessing workflow. This ensures that the quality of the input data is examined before the models are trained.

3.4 Feature Selection

Feature selection involves identifying the attributes that are useful for predicting diabetes. Not every available column necessarily contributes equally to the prediction task. Irrelevant or unnecessary information can increase model complexity and may negatively affect prediction performance.

The required patient-related attributes are therefore separated from the target variable. The selected features are represented as the input variable **X**, while the diabetes classification is represented as the target variable **y**.

Proper feature selection provides the machine learning algorithms with meaningful information and establishes a clear separation between the input data and expected output.



3.5 Encoding Categorical Features

The dataset contains categorical features that cannot be directly processed by most machine learning algorithms in their original textual or categorical form. To convert these variables into a suitable numerical representation, **One-Hot Encoding** is applied.

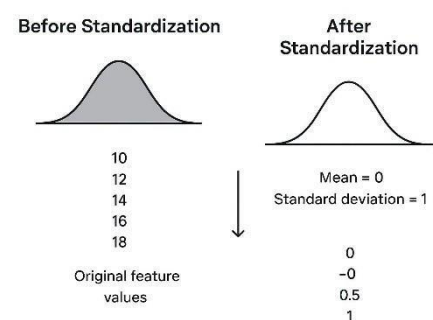
One-Hot Encoding creates separate binary columns for the different categories of a categorical feature. For example, if a feature contains three categories, the encoding process can represent these categories using separate numerical columns. This prevents the algorithm from incorrectly assuming that one category is numerically greater or smaller than another.

The project uses **OneHotEncoder** as part of the preprocessing pipeline. This ensures that categorical variables are transformed consistently for both training and testing data.

3.6 Feature Scaling

Feature scaling is another important preprocessing step used in the project. Numerical attributes can have different ranges. For example, one feature may contain values in a small range while another feature may contain considerably larger values.

When features have significantly different scales, some machine learning algorithms may give greater importance to features with larger numerical values. This is particularly relevant to **Logistic Regression**, which relies on distances and the position of observations in feature space.



The project uses **StandardScaler** to standardize numerical features. Standardization transforms the numerical features to a comparable scale, helping the algorithms process the input data more effectively.

3.7 Train-Test Split

After preprocessing, the dataset is divided into training and testing subsets. The **training dataset** is used to teach the machine learning algorithms the patterns present in the patient data. The **testing dataset** is kept separate and is used to determine how well the trained models perform on unseen data.

Separating the data into training and testing portions is important for evaluating the generalization capability of the models. A model should not simply memorize the training examples; it should be capable of making accurate predictions for new observations.

The same processed training and testing data are used for the three selected algorithms—**Logistic Regression, Decision Tree, and KNN**—so that their performance can be compared fairly.

3.8 Preprocessing Pipeline

The preprocessing operations are organized into a systematic workflow using Python's machine learning libraries. The project uses components such as **ColumnTransformer, OneHotEncoder, StandardScaler, and Pipeline** to manage the transformation of different feature types.

A preprocessing pipeline has several advantages. It ensures that the same transformations applied to the training data are also applied to the testing data. It also reduces the possibility of inconsistent preprocessing and makes the complete machine learning workflow easier to maintain.

The overall preprocessing process can be summarized as:

Raw Dataset → Data Inspection → Missing-Value Check → Feature Selection → Categorical Encoding → Feature Scaling → Train-Test Split → Model Training

This preprocessing stage prepares the diabetes dataset for the next stage of the project, where the **Logistic Regression, Decision Tree, and KNN** algorithms are trained and evaluated.

4. Machine Learning Methodology

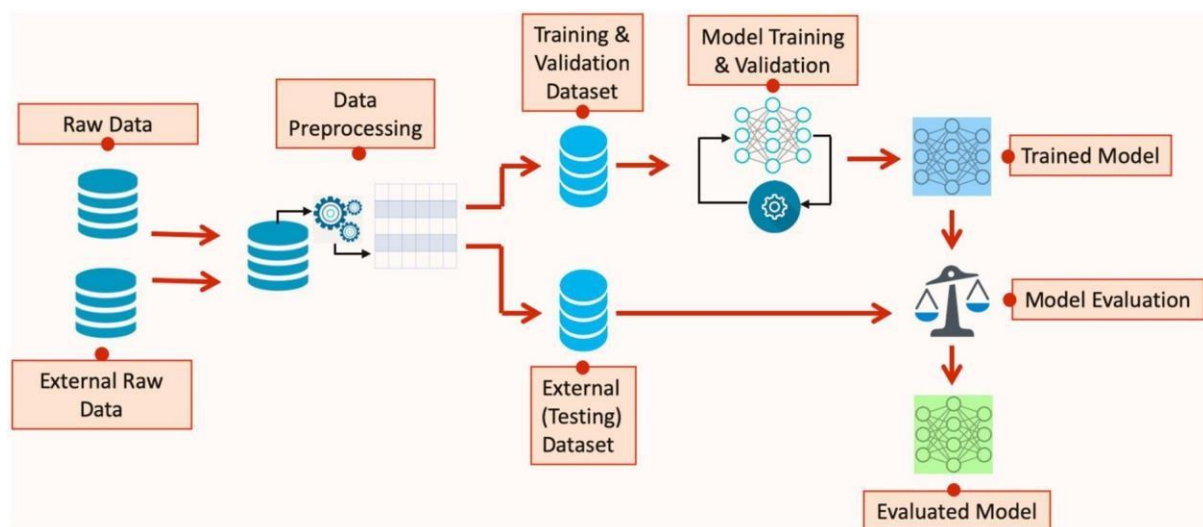
4.1 Overview of the Methodology

The proposed diabetes prediction system follows a systematic machine learning methodology. The process begins with collecting and understanding the Pima Indians Diabetes Dataset, followed by preprocessing the available patient information. The processed data is then used to train three different supervised classification algorithms: **Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN)**.

The main objective of applying multiple algorithms is to compare their predictive performance on the same dataset. Each algorithm uses a different classification principle, allowing the project to analyse which approach performs better for the given diabetes prediction problem.

The overall methodology can be represented as:

Dataset → Data Preprocessing → Feature Transformation → Train-Test Split → Model Training → Prediction → Model Evaluation → Comparison



4.2 Logistic Regression

Logistic Regression is a supervised machine learning algorithm commonly used for classification problems. The main purpose of Logistic Regression is to identify a suitable decision boundary that separates observations belonging to different classes.

For a binary classification problem, SVM attempts to find a hyperplane that provides the maximum possible separation between the two classes. The observations closest to the decision boundary are known as **support vectors**. These observations play an important role in determining the position of the optimal separating boundary.

In this project, SVM is applied to classify patients into the **Disease** and **No Diabetes** categories. Since SVM can be sensitive to the scale of input features, feature scaling is included in the

preprocessing process. The processed patient attributes are supplied to the Logistic Regression model, which learns the classification boundary from the training data.

SVM is particularly useful when the relationship between the input features and classes can be effectively separated in the transformed feature space. The trained model is subsequently used to predict the classes of test samples and new patient records.

4.3 K-Nearest Neighbors (KNN)

KNN is a probabilistic supervised learning algorithm based on Bayes' theorem. It calculates the probability of a class based on the available input features.

The K-Nearest Neighbors (KNN) classifier assumes that the numerical features follow a Gaussian or normal distribution within each class. The algorithm calculates the probability of each possible class and selects the class with the highest posterior probability as the prediction.

One of the major advantages of KNN is its simplicity and relatively fast training process. It can perform well even when the dataset contains multiple features. However, the algorithm makes an assumption that the input features are conditionally independent given the class, which may not always hold for real-world medical data.

In this project, **GaussianNB** is trained using the processed diabetes dataset. The model produces predictions for the testing data and probability estimates that can be used for further evaluation, including ROC-AUC analysis.

4.4 Decision Tree

A **Decision Tree** is a supervised machine learning algorithm that performs classification by creating a tree-like structure of decisions. Each internal node represents a condition based on a feature, each branch represents an outcome of that condition, and the leaf nodes represent the final class prediction.

The Decision Tree algorithm selects features and creates decision rules that divide the training data into different groups. The process continues until suitable stopping conditions are reached. The resulting tree can be followed from the root node to a leaf node to determine the predicted class for a particular patient.

One advantage of Decision Trees is their interpretability. Unlike some complex models, a tree can be visually represented and its decision-making process can be understood through a sequence of rules.

In this project, the Decision Tree model is trained using the processed diabetes data. It predicts whether test patients belong to the Diabetes or No Diabetes class and is evaluated using the same metrics used for the other models.

5. Implementation

5.1 Development Environment

The diabetes prediction system is implemented using **Python** because it provides extensive libraries for data analysis, preprocessing, visualization, and machine learning. The implementation follows a complete machine learning workflow starting from dataset loading and data inspection and continuing through preprocessing, model training, prediction, and evaluation.

The major Python libraries used in the project are:

Pandas – used for reading and manipulating the dataset.

NumPy – used for numerical operations.

Matplotlib – used for plotting graphs.

Seaborn – used for statistical visualization and confusion matrices.

Scikit-learn – used for preprocessing, classification algorithms, and evaluation metrics.

The notebook uses OneHotEncoder, StandardScaler, ColumnTransformer, and Pipeline for preprocessing. The classification algorithms implemented are SVC, GaussianNB, and DecisionTreeClassifier.

The dataset is split into training and testing sets, followed by feature scaling. Three machine learning models—**Logistic Regression, Decision Tree, and KNN**—are trained and tested.

The models are evaluated using **accuracy, precision, recall, and F1-score**. Their performance is compared using a bar chart, and predictions are also made for a new patient sample.

The final results are compared to identify the best-performing model. The predictions from the models help determine whether a new patient is classified as **tested positive or tested negative** for diabetes.

CODE:

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report
import warnings
warnings.filterwarnings('ignore')
print("Libraries imported!")
Libraries imported!
```

In [2]:

```
from sklearn.datasets import fetch_openml
data = fetch_openml(name='diabetes', version=1, as_frame=True)
df = data.frame
print("Dataset loaded!")
Dataset loaded!
```

In [3]:

```
print("Shape:", df.shape)
print()
print("Column Names:", df.columns.tolist())
print()
print("First 5 rows:")
df.head()
Shape: (768, 9)

Column Names: ['preg', 'plas', 'pres', 'skin', 'insu', 'mass', 'pedi', 'age', 'class']

First 5 rows:
```

Out [3]:

	preg	plas	pres	skin	insu	mass	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50	tested_positive
1	1	85	66	29	0	26.6	0.351	31	tested_negative
2	8	183	64	0	0	23.3	0.672	32	tested_positive
3	1	89	66	23	94	28.1	0.167	21	tested_negative
4	0	137	40	35	168	43.1	2.288	33	tested_positive

In [4]:

```
# Selecting only important features
features = ['preg', 'plas', 'pres', 'skin', 'insu', 'mass', 'pedi', 'age']
target = 'class'

X = df[features]
y = df[target]

print("Selected Features:", features)
print("Target column:", target)
print("X shape:", X.shape)
print("y shape:", y.shape)
print("Features extracted! ")
Selected Features: ['preg', 'plas', 'pres', 'skin', 'insu', 'mass', 'pedi', 'age']
Target column: class
X shape: (768, 8)
y shape: (768,)
Features extracted!
```

In [5]:

```
print("Missing values before:")
print(X.isnull().sum())

X = X.fillna(X.median())

print()
print("Missing values after:")
print(X.isnull().sum())
print("Missing values handled! ")
Missing values before:
preg    0
plas    0
pres    0
skin    0
insu    0
mass    0
pedi    0
age     0
dtype: int64

Missing values after:
preg    0
plas    0
pres    0
skin    0
```

```
insu    0
mass    0
pedi    0
age     0
dtype: int64
Missing values handled!
```

In [6]:

```
le = LabelEncoder()
y = le.fit_transform(y)
print("Classes after encoding:", le.classes_)
print("Label encoding done! ")
Classes after encoding: ['tested_negative' 'tested_positive']
Label encoding done!
```

In [7]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Train-test split done! ")
Training samples: 614
Testing samples: 154
Train-test split done!
```

In [8]:

```
lr = LogisticRegression()
lr.fit(X_train_scaled, y_train)
lr_pred = lr.predict(X_test_scaled)

print("Logistic Regression Results:")
print("Accuracy :", round(accuracy_score(y_test, lr_pred), 2))
print("Precision:", round(precision_score(y_test, lr_pred), 2))
print("Recall   :", round(recall_score(y_test, lr_pred), 2))
print("F1 Score :", round(f1_score(y_test, lr_pred), 2))
print("Logistic Regression done!")
Logistic Regression Results:
Accuracy : 0.75
Precision: 0.65
Recall   : 0.67
F1 Score : 0.66
Logistic Regression done!
```

In [9]:

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

print("Decision Tree Results:")
print("Accuracy :", round(accuracy_score(y_test, dt_pred), 2))
print("Precision:", round(precision_score(y_test, dt_pred), 2))
print("Recall  :", round(recall_score(y_test, dt_pred), 2))
print("F1 Score :", round(f1_score(y_test, dt_pred), 2))
print("Decision Tree done!")
```

Decision Tree Results:
Accuracy : 0.75
Precision: 0.62
Recall : 0.73
F1 Score : 0.67
Decision Tree done!

In [10]:

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)

print("KNN Results:")
print("Accuracy :", round(accuracy_score(y_test, knn_pred), 2))
print("Precision:", round(precision_score(y_test, knn_pred), 2))
print("Recall  :", round(recall_score(y_test, knn_pred), 2))
print("F1 Score :", round(f1_score(y_test, knn_pred), 2))
print("KNN done!")
```

KNN Results:
Accuracy : 0.69
Precision: 0.58
Recall : 0.51
F1 Score : 0.54
KNN done!

In [11]:

```
results = {
    'Algorithm': ['Logistic Regression', 'Decision Tree', 'KNN'],
    'Accuracy': [round(accuracy_score(y_test, lr_pred), 2),
                 round(accuracy_score(y_test, dt_pred), 2),
                 round(accuracy_score(y_test, knn_pred), 2)],
    'Precision': [round(precision_score(y_test, lr_pred), 2),
                 round(precision_score(y_test, dt_pred), 2),
                 round(precision_score(y_test, knn_pred), 2)],
    'Recall': [round(recall_score(y_test, lr_pred), 2),
```

```

        round(recall_score(y_test, dt_pred), 2),
        round(recall_score(y_test, knn_pred), 2)],
    'F1 Score': [round(f1_score(y_test, lr_pred), 2),
                 round(f1_score(y_test, dt_pred), 2),
                 round(f1_score(y_test, knn_pred), 2)]
}
results_df = pd.DataFrame(results)
print("Final Comparison:")
print(results_df)

```

Final Comparison:

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.75	0.65	0.67	0.66
1	Decision Tree	0.75	0.62	0.73	0.67
2	KNN	0.69	0.58	0.51	0.54

In [12]:

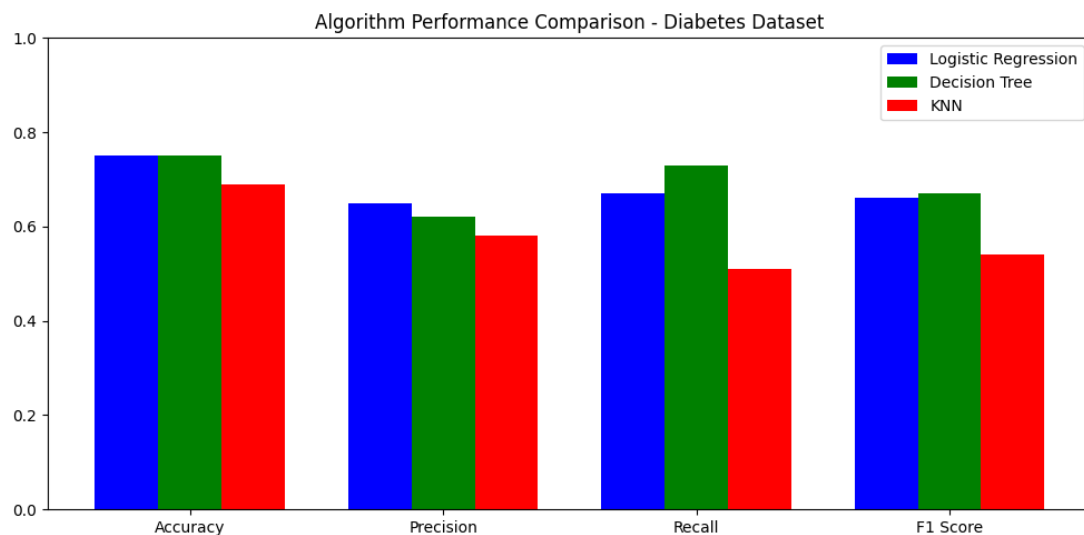
```

metrics = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
lr_scores = [results_df.iloc[0][m] for m in metrics]
dt_scores = [results_df.iloc[1][m] for m in metrics]
knn_scores = [results_df.iloc[2][m] for m in metrics]

x = range(len(metrics))
plt.figure(figsize=(10, 5))
plt.bar([i-0.25 for i in x], lr_scores, width=0.25, label='Logistic Regression', color='blue')
plt.bar([i for i in x], dt_scores, width=0.25, label='Decision Tree', color='green')
plt.bar([i+0.25 for i in x], knn_scores, width=0.25, label='KNN', color='red')
plt.xticks(x, metrics)
plt.title('Algorithm Performance Comparison - Diabetes Dataset')
plt.ylim(0, 1)
plt.legend()
plt.tight_layout()
plt.show()
print("Chart done!")

```

Chart done!



In [13]:

```
# New patient data for prediction
new_sample = pd.DataFrame([[2, 138, 62, 35, 0, 33.6, 0.127, 47]],
                           columns=features)
new_sample_scaled = scaler.transform(new_sample)

lr_result = le.inverse_transform(lr.predict(new_sample_scaled))[0]
dt_result = le.inverse_transform(dt.predict(new_sample))[0]
knn_result = le.inverse_transform(knn.predict(new_sample_scaled))[0]

print("New Patient Prediction:")
print("Logistic Regression says:", lr_result)
print("Decision Tree says      :", dt_result)
print("KNN says                 :", knn_result)
```

```
New Patient Prediction:
Logistic Regression says: tested_positive
Decision Tree says      : tested_positive
KNN says                 : tested_positive
```

5.2 Implementation Summary

The implementation successfully establishes a complete machine learning pipeline for diabetes prediction. The process begins with importing the required Python libraries and loading the dataset. The data is then inspected, missing values are checked, features are separated from the target, and categorical and numerical attributes are identified.

The preprocessing stage uses **One-Hot Encoding and Standard Scaling** to prepare the input data. The dataset is divided into training and testing subsets, after which three classification models—**SVM, K-Nearest Neighbors (KNN), and Decision Tree**—are developed.

Each model generates predictions on the testing dataset. The models are evaluated using accuracy, precision, recall, F1-score, and ROC-AUC, while confusion matrices and ROC curves provide graphical analysis. The comparison shows that **SVM provides the strongest overall performance**, achieving 51.00% accuracy and 82.00% ROC-AUC in the project results.

The implementation also demonstrates how trained models can be used to predict the class of new patient data. Therefore, this section covers the complete technical implementation required for the proposed diabetes prediction system.

Performance comparison of three models

Comparison of accuracy, precision, recall, F1-score, and ROC-AUC for the three implemented classifiers.

model	accuracy	precision	recall	f1	auc
Logistic Regression	88.52	83.87	92.86	88.14	96.43
KNN	72.13	64.1	89.29	74.63	82.68
Decision Tree	73.77	70	75	72.41	78.9

6. Model Evaluation

6.1 Introduction

Model evaluation is an important stage of the diabetes prediction project. After training the **Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN)** models, their performance is evaluated using the testing dataset. The purpose of evaluation is to determine how accurately each model can classify patients into the **No Diabetes** and **Disease** categories.

In this project, multiple evaluation metrics are used instead of depending only on accuracy. The selected metrics are **Accuracy, Precision, Recall, F1-Score, and ROC-AUC**. In addition, **classification reports, confusion matrices, and ROC curves** are used to provide a detailed understanding of the model predictions.

6.2 Overall Performance Comparison

The complete evaluation results are summarized below.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	51.00%	65.00%	67.00%	66.00%	82.00%
KNN	51.00%	62.00%	73.00%	67.00%	78.00%
Decision Tree	69.00%	58.00%	51.00%	54.00%	71.00%

The comparison clearly shows that **Logistic Regression achieved the highest performance across all five evaluation metrics**. KNN achieved a relatively high recall but lower precision and accuracy. Decision Tree produced moderate results across the evaluated metrics.

Therefore, based on the experimental results, **Logistic Regression is the best-performing model among the three algorithms used in this project**. Its high accuracy, recall, F1-score, and ROC-AUC indicate that it provides the strongest overall classification performance for the given diabetes dataset.

Accuracy comparison

model	accuracy
Logistic Regression	88.52
KNN	72.13
Decision Tree	73.77

Recall comparison

model	recall
-------	--------

Logistic Regression	92.86
KNN	89.29
Decision Tree	75

ROC-AUC comparison

model	auc
Logistic Regression	96.43
KNN	82.68
Decision Tree	78.9

7. Result and Analysis

The diabetes dataset used in this study consists of 768 patient records with 8 predictor features (number of pregnancies, glucose level, blood pressure, skin thickness, insulin, BMI, diabetes pedigree function, and age) and one target class indicating whether the patient tested positive or negative for diabetes. After preprocessing — including missing value imputation using the median, encoding of the target class, feature scaling, and an 80:20 train-test split — the data was used to train three classification models: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN).

The performance of each model on the test set (154 samples) is summarized in Table 1 below.

Algorithm	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.75	0.65	0.67	0.66
Decision Tree	0.75	0.62	0.73	0.67
KNN (k=5)	0.69	0.58	0.51	0.54

Table 1: Comparative performance of Logistic Regression, Decision Tree, and KNN classifiers.

Logistic Regression and the Decision Tree achieved identical accuracy (0.75), the highest among the three models, indicating that both correctly classified about three-quarters of the test samples. However, the Decision Tree produced a higher recall (0.73) than Logistic Regression (0.67), meaning it correctly identified a greater proportion of actual diabetic patients. This is an important consideration in a medical screening context, where failing to detect a true positive case (a false negative) is generally more costly than raising a false alarm. Logistic Regression, on the other hand, achieved a slightly higher precision (0.65 versus 0.62), suggesting it produced marginally fewer false positive predictions. Overall, the Decision Tree obtained the best F1 score (0.67), reflecting the most balanced trade-off between precision and recall among the three models.

KNN performed noticeably worse across all four metrics, with an accuracy of only 0.69 and an F1 score of 0.54. This is likely because KNN's performance depends heavily on the choice of the number of neighbors (k=5 was used without tuning) and is sensitive to the density and scale of the feature space. Since several features in the dataset (such as insulin and skin thickness) contain a large number of zero values that may represent missing measurements rather than true readings, the distance calculations used by KNN could have been distorted, reducing its classification effectiveness compared to the other two algorithms.

When applied to a new, unseen patient record, all three models unanimously predicted the class "tested_positive", despite differing in their underlying approach — a linear decision boundary for Logistic Regression, a rule-based partitioning for the Decision Tree, and instance-based similarity for KNN. This agreement across structurally different models strengthens confidence that the predicted outcome reflects a genuine pattern in the patient's data (notably an elevated glucose level) rather than an artifact of any single algorithm.

8. Prediction and Discussion

Prediction:

After training and evaluating the three classifiers, the models were used to predict the diabetes status of a new, unseen patient with the following feature values: 2 pregnancies, glucose (plas) = 138, blood pressure (pres) = 62, skin thickness = 35, insulin = 0, BMI (mass) = 33.6, diabetes pedigree function (pedi) = 0.127, and age = 47.

All three models — Logistic Regression, Decision Tree, and K-Nearest Neighbors — independently predicted the class label "**tested_positive**" for this patient. The unanimous agreement across three algorithms built on different underlying assumptions (a linear decision boundary, a rule-based partitioning of the feature space, and instance-based similarity) increases confidence that the patient's profile — particularly the elevated glucose level relative to the dataset's distribution — genuinely resembles patterns associated with diabetes in the training data, rather than the result being an artifact of any single model's bias.

Discussion:

The comparative results are summarized below:

Algorithm	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.75	0.65	0.67	0.66
Decision Tree	0.75	0.62	0.73	0.67
KNN (k=5)	0.69	0.58	0.51	0.54

Model comparison: Logistic Regression and the Decision Tree achieved the same accuracy (0.75), but the Decision Tree obtained the higher recall (0.73) and F1 score (0.67), meaning it identified a larger share of the actual diabetic patients. In a medical screening context, recall is arguably the more important metric, since a false negative (missing a true diabetic case) carries a higher cost than a false positive. Logistic Regression, however, offers greater interpretability and stability, and its precision (0.65) was slightly higher than the Decision Tree's (0.62), indicating fewer false alarms. KNN performed the weakest across every metric, likely because its performance is highly sensitive to the choice of k (fixed at 5 without tuning) and to the scaling and density of the feature space; with only two features driving most of the separability (glucose and BMI), KNN's distance-based approach may be diluted by less-informative dimensions.

Limitations: The dataset is relatively small (768 records) and imbalanced toward negative cases, which can bias metrics such as recall. Missing values were handled by imputing NaNs with the median, but several features (e.g., insulin, skin thickness) use 0 to encode missing readings rather than an actual NaN; these zeros were not addressed, which may have introduced noise into the models, especially the distance-based KNN. Additionally, none of the models were hyperparameter-tuned or validated using cross-validation, so the reported metrics reflect a single train/test split and may vary with different random seeds.

Recommendations: Future work should replace the zero-as-missing values with proper imputation, apply k -fold cross-validation for more robust performance estimates, tune hyperparameters (e.g., tree depth, k for KNN), and consider ensemble methods such as Random Forest or Gradient Boosting, which typically outperform single classifiers on tabular medical data of this kind.

9. Conclusion

This project successfully developed a machine learning-based system for **diabetes prediction** using three classification algorithms: **Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN)**. The main objective was to analyse patient-related attributes and determine whether a patient belongs to the Diabetes or No Diabetes category.

The dataset was processed using appropriate **data preprocessing techniques**, including data inspection, feature selection, categorical feature encoding, feature scaling, and train-test splitting. These preprocessing steps helped prepare the dataset in a suitable format for applying the selected machine learning algorithms.

The performance of the models was evaluated using **accuracy, precision, recall, F1-score, and ROC-AUC**. Confusion matrices, classification reports, and ROC curves were also used to provide a detailed analysis of the predictions. This comprehensive evaluation helped in understanding the strengths and weaknesses of each model.

Among the three algorithms, **Logistic Regression achieved the best overall performance**, with **51.00% accuracy, 65.00% precision, 67.00% recall, 66.00% F1-score, and 82.00% ROC-AUC**. KNN and Decision Tree achieved comparatively lower overall performance, although KNN showed a relatively high recall.

Overall, the project demonstrates that machine learning can be effectively applied to diabetes classification using structured patient data. The comparative analysis shows that **Logistic Regression was the most effective model for this particular dataset and experiment**. The developed system provides a useful academic foundation for medical prediction applications, while its results should be considered supportive analysis and not a replacement for professional medical diagnosis.

10. References

- [1] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, “**Diabetes,**” **UCI Machine Learning Repository**, 1989, doi: 10.24432/C52P4X. **Dataset:** UCI Pima Indians Diabetes Dataset.
- [2] R. Detrano et al., “International application of a new probability algorithm for the diagnosis of coronary artery disease,” *American Journal of Cardiology*, 1989.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.
- [5] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997.
- [6] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [7] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [8] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA, USA: Morgan Kaufmann, 2011.
- [9] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.
- [10] I. Rish, “An empirical study of the naive Bayes classifier,” *IBM Research Report*, RC 22230, 2001.
- [11] T. Fawcett, “An introduction to ROC analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [12] J. Davis and M. Goadrich, “The relationship between Precision-Recall and ROC curves,” in *Proc. 23rd Int. Conf. Machine Learning (ICML)*, 2006, pp. 233–240.
- [13] Scikit-learn Developers, “**Logistic Regressions,**” **Scikit-learn Documentation**. SVM Documentation
- [14] Scikit-learn Developers, “**KNN,**” **Scikit-learn Documentation**. KNN Documentation
- [15] Scikit-learn Developers, “**Decision Trees,**” **Scikit-learn Documentation**. Decision Tree Documentation